MicroApp Development Guide

- Conceptualizing your MicroApp
- MicroApp Rules
- Machine Setup
  - Setting up your IDE
  - Pre-Requisites
  - Setting Up NPM
    - On ZScaler Private Access & Authenticated
    - Not On ZScaler Type
- Development
  - Create your MicroApp
  - Anatomy of a MicroApp
  - Running your MicroApp
  - Developing your MicroApp
    - Remove Example Code
    - Update .clinerules/03-tool-schemas.md
    - Tell Cline what you want
    - Example
    - Testing Your Code
    - Running your Code
- Deploying your MicroApp
  - Additional Help
  - Supporting your MicroApp
- Updating the MicroApp Framework
- Rate Limits
- Working with Snowflake Data
  - Enabling Snowflake
  - snowflake.yaml
    - Bulk Loading
      - Important: Parameter Syntax Conflicts
      - Example: Bulk Insert/Update Query
      - Example: TypeScript Implementation
    - Incremental Fetching Data
  - Schema Management
  - Backup/Restore
  - Monitoring
  - Data Security
- Working with Postgres Data Store
  - Enabling Data Store
  - Working with Data (Simple Mode)
  - Working with Data (Advanced Mode)
  - data-store.yaml
  - Schema Management
  - Backup/Restore
  - Performance
  - Monitoring
  - Data Security
- Getting Help

Troubleshooting
NPM
WITH Zscaler Private Access
WITHOUT Zscaler Private Access
Developer Access Not Working

Chat interfaces provide a valuable way to interact with AI and data, but they aren't always the most efficient solution for specific workflows. MicroApps are focused mini-applications that offer users a more streamlined interface than traditional chat boxes for completing targeted tasks. By design, MicroApps maintain a very limited scope, concentrating on solving specific problems rather than attempting broad functionality. They're built to be easily implementable through intuitive development approaches, enabling non-technical users to create AI-powered tools that enhance their productivity and workflow efficiency.

Conceptualizing your MicroApp

The most important step in creating a MicroApp is determining what it is you're trying to accomplish. Sketch out what you want to build. What does it look like? How would a user interact with it? Do we have existing tools that can get the data you need? Is what you're trying to do something that can be done via a chat box? Perhaps an Agent is a better choice for you? If you're new to this, maybe ask for help in the `#zi-chat-microapps` channel

MicroApp Rules

Before you start building your MicroApp, make sure what you are trying to do follows these rules:

- Can only call existing ZI Chat tools and execute prompts.
- Cannot access external URLs, local storage, cookies
- Only consist of front-end (UI) code.

If you can't make your plan fit into these rules, perhaps additional MCP tools need to be added to ZI Chat, or there is a need to make something bigger than a MicroApp.

Machine Setup

Great. You have an idea for a MicroApp. We have the tools you need to build it. Now it's time to jump in!

Setting up your IDE

An IDE is a tool that helps you write code. Unless you are familiar with other IDEs, we strongly recommend that you use vscode to build microapps. Here is a list of things you should install.

Specific instructions are beyond the scope of this document, but there are plenty of resources that can help you with this. A very large part of development is trying to figure out problems on your own. If you get stuck, ask the ZI Chat for help (search for any error message you're seeing)! If that doesn't work `#zi-chat-microapps`.

Pre-Requisites

1. ZScaler (ZPA & ZPI) - the corporate VPN is required to access our staging environment, but is not required in the product environment. If you don't have this on your company laptop, you can request it here: Login - Service Portal ZoomInfo
2. Install VS Code - VS Code is an IDE for writing the code. We are using in this document for writing node js app.
3. Install the Cline VS Code extension - Cline VS Code is a AI plugin which assists with writing code in VS Code. Request an Anthropic API key via this Slack workflow: https://zoominfo.enterprise.slack.com/archives/C092UAZ4LS0 Connect your Slack account
4. Install Node.js - This is needed to create a node project. Its an SDK for node projects.
5. `npm login` with JFrog (put in a ticket to get access) Troubleshooting
   a. This is needed to pull the dependencies from JFROG Artifactory.
6. Install Git so you can easily share your work, keep safe copies of everything, and go back to earlier versions if needed. (optional)

Setting Up NPM

Be sure NPM is set up. If you've never set it up before on your machine, you can run a command to set it up globally on your machine.

On ZScaler Private Access & Authenticated

```
npm set registry https://artifacts.zi-int.com/artifactory/api/npm/zi-npm-virtual/
```

Not On ZScaler Type

```
npm set registry https://zoominfo.jfrog.io/artifactory/api/npm/virtual-npm/

npm login --auth-type=web
```

Development

Now that your machine is ready to go, it's time to start working on building your micro app. Developing a micro app consists of 3 broad steps.

- Creating the MicroApp (adds folders/files that you need to get started to your machine)

- Developing the MicroApp (the fun part where you make the app do what you want)
- Deploying the MicroApp (packaging it up so others can use it)

Create your MicroApp

Now that you have an environment set up, you are ready to create your first micro app.

1. Go to the terminal and type. For executing this command, you should have completed Step 4 from Pre-requisites.

```
npx @zoominfo-chat/create-micro-app <your-micro-app-name>
```

> If the above command doesn't work for you, you may need to add a npm registry path (such as the one below)
>
> --registry=https://artifacts.zi-int.com/artifactory/api/npm/zi-npm-virtual/

2. Assuming you have set everything up you should see something like the below

```
🚀 Creating ZoomInfo Micro-App...
✔ Project name: … your-micro-app
📁 Created directory: your-micro-app
📄 Created template files

✅ Micro-app created successfully!

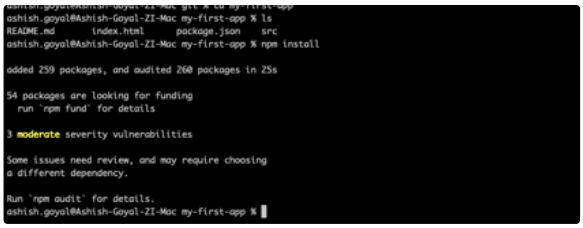
Next steps:
  cd your-micro-app # navigate to your project directory
  npm install # to install dependencies
  code . # to open in VSCode
  npm run dev # when you want to start your app
```

3. Go to your micro app folder.

```
cd your-micro-app  # this will change your directory to the micro-app directory
```

4. Install npm to install dependencies.

```
npm install # this will install dependencies
```

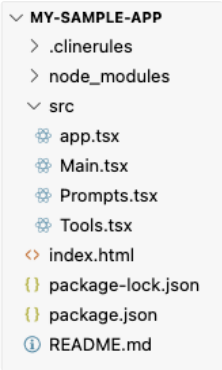


5. To open the current directory in VS code you can run. If this command does not work, then you can directly open VS Code IDE on your system.

```
code .
```

Anatomy of a MicroApp

A new MicroApp application consists of quite a lot of files. This section tells you all about them.



When you open your new folder in vscode you should see something similar to the above

File/Directory	Description
.clinerules	The files in this directory help Cline know how to work with the repo.
node_modules (if missing call `npm install`)	This is a folder that contains source code for all the packages you import. You can also typically ignore this folder.
.server	These files pre-define queries that can be run through the UI for applications that use data storage.
.npmrc	An .npmrc file is a configuration file for **npm** (Node Package Manager). It lets you define settings that control how npm behaves when installing, publishing, or managing packages.

	You can think of it as a “preferences” file for npm.
src/app.tsx	This is the main “bootstrap” file that launches your micro app (by loading the `Main` component). This file may be changed in future releases so it’s best you never change this file.
src/Main.tsx	This is the main entry point where you will put all your code. When your initial app is created this contains a simple tabbed interface.
src/Prompts.tsx	This component shows you how to call the prompt-related APIs. This will allow you to add “AI” to your tool. We ship both a sync (make a call, wait until it completes) and a streaming (show the text as it comes back) API. The streaming API is more difficult to work with, but has a better user experience.
src/Tools.tsx	This component shows you how to interact with various tools. You can click on an individual tool, enter some data in JSON or a form, and then call the tool and see the results. There are may be few bugs with the web form.
src/ZoomInfoApi.tsx	This component shows you how to interact with internal ZoomInfo HTTP APIs.
src/Agents.tsx & src/AgentExecutionResults.tsx	These components show you how to interact with agents and the data

	that is returned by an agent call. You can click on an individual agent, enter some data in JSON or a form, and then call the agent and see the results. There are may be few bugs with the web and display.
src/DataStorage.tsx	This component shows you how to interact with various data storage APIs by implementing a very simple to-do list.
src/ExecutionContext.tsx	This component shows you how to interact with the execution context (environment, user information, timezone, locale, etc).
src/MicroApps.tsx	This component shows you how to interact with with various micro app APIs.
src/Users.tsx	This component shows you how to get a list of all available ZI Chat users.
index.html	This is the first file that is downloaded when a MicroApp is loaded. You shouldn't touch this file unless absolutely necessary.
package-lock.json (may be missing)	This file contains the exact versions of the packages you want to load. This is a file that is created/updated by various npm commands such as `npm install`.
package.json	This is a manifest file that contains information about the various dependencies of your application along with some helpful scripts that

	you will use to run and deploy your app.

Running your MicroApp

Great! You've created your first micro-app. This section will show you how to run it in "dev" mode. When you're running in dev mode you can make changes to your application and see them in your browser. If you have multiple monitors, it's a good idea to put VSCode on one, and your browser window on the other!

To run your app in dev mode. Do the following:

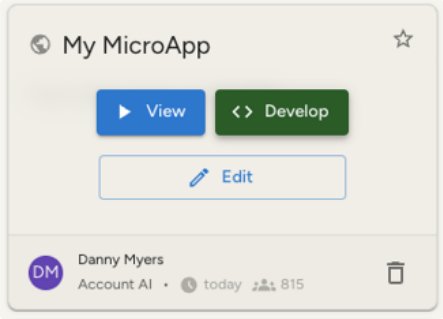
1. Go to your terminal and type

```
npm run dev
```

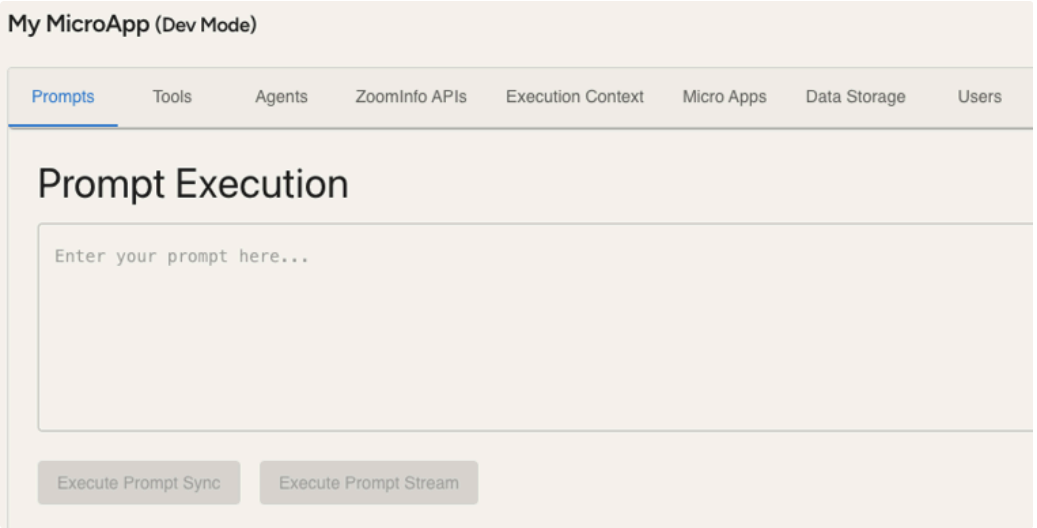
2. You should see something like below

```
🚀 Starting micro-app development server...
✅ Development server running at http://localhost:5173
```

3. Navigate to ZI Chat's production environment and open MicroApps: ZI Chat
4. Create a new micro app (we recommend that you keep it private until you're ready for others to use it).
5. Click on "Develop"



6. You should see something similar to the below micro-app



7. Make some changes to the app. Try to find where it says “Prompt Execution” in a header and rename it to something else. Hit save and you should see the changes immediately reflected in the UI. 🪄

Developing your MicroApp

⚠️ Warning - writing code can be addictive. A lot of us love doing this all day! 🙂

Great – you’ve come this far. That means you are ready to build something amazing. What should you do? If you’re new to development, we recommend you try vibe-coding your way to success (at least to start). This means you’re going to be chatting with the AI to help you write code.

Remove Example Code

Before you start vibing though, you probably don’t need the sample app files anymore. Go ahead and remove them:

- Delete everything in `/src` except `app.tsx` and `Main.tsx`
- Replace Main.tsx with the below

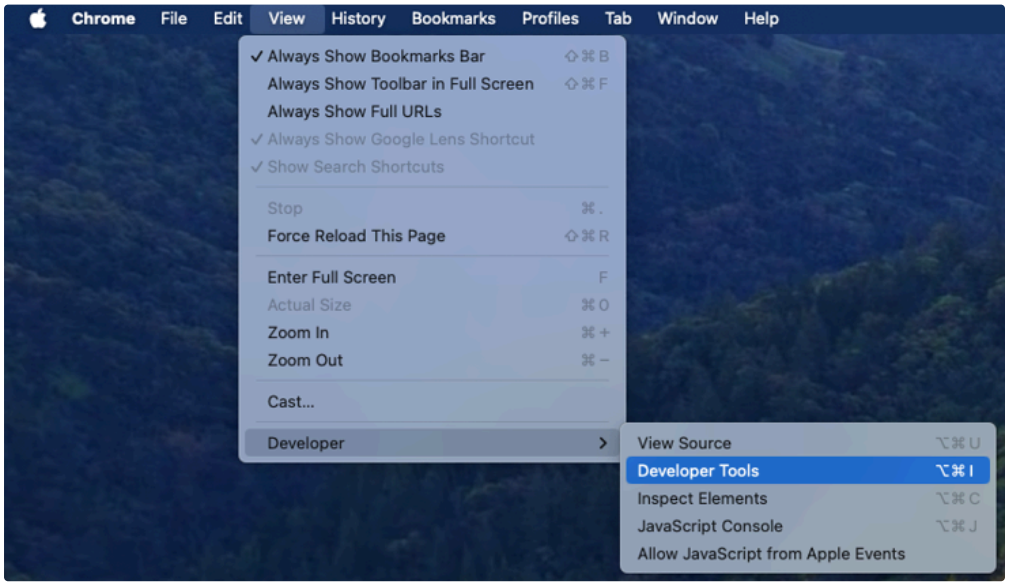
```
export default function Main() {
  return <div>Hello</div>
}
```

If you do the above while your app is running (npm run dev), you should see changes instantly reflected in the micro app frame.

Update .clinerules/03-tool-schemas.md

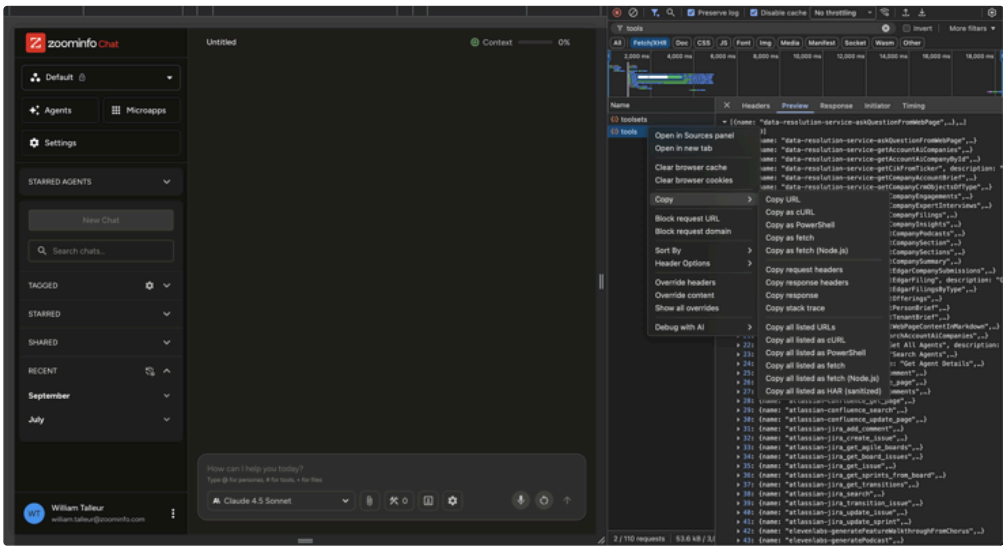
Cline will work much better if it understands what tools it can access, how to call them, and what results are returned. Not every MicroApp developer has access to every type of tool, so it makes more sense for you to capture the tools that are available to you, and paste them in this file.

- Open your browser's debug tools.



Open Developer Tools in your favorite browser

- Navigate to ZI Chat
- Select the workspace where your micro-app will reside. (Be sure you have connected any integrations you may want to use.)
- Copy the response from the tools call.



- Open `.clinerules/03-tool-schemas.md` in your IDE

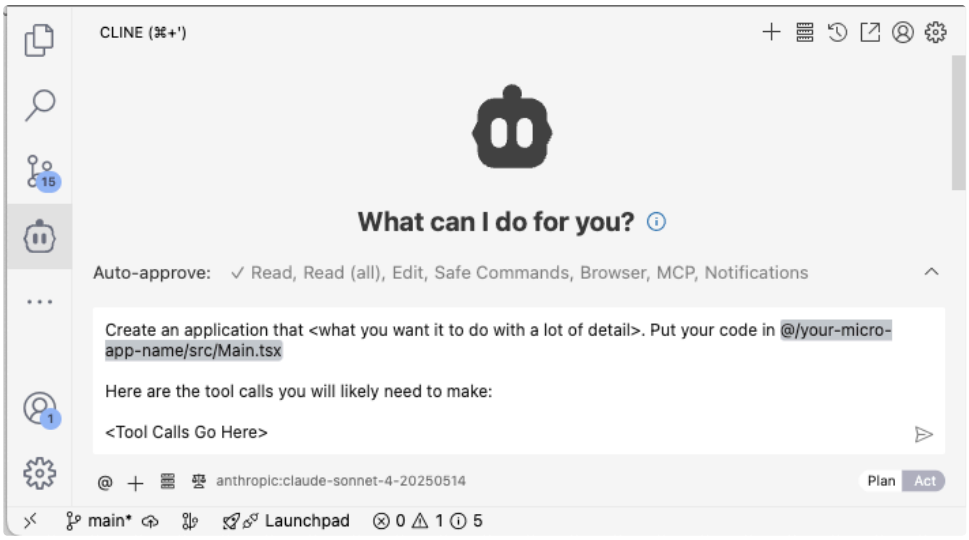
- Paste the tools response.

```
##### Available Tools

Below is a list of all available tools along with their input and output schemas:

```json
[{"the tool definitions": "go here"}]
```
```

Tell Cline what you want



Cline will then automatically write a ton of code for you. You will likely need to iterate on it multiple times until you get an output you like.

Example

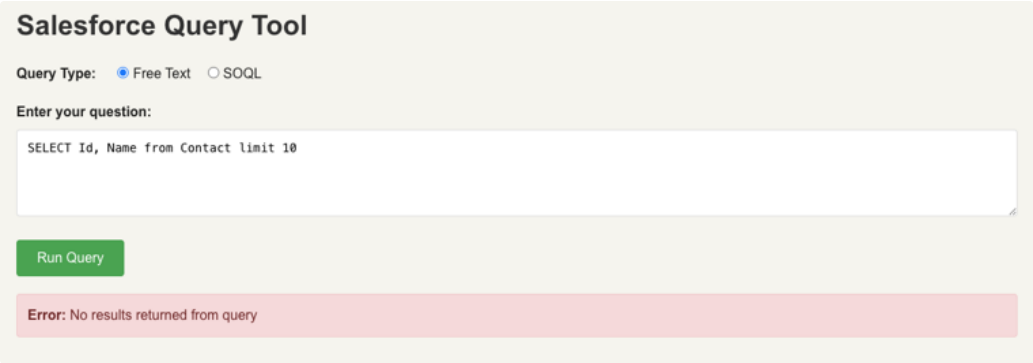
Below is an example based on an early build of MicroApps. If you put the tool schemas in `.clinerules/03-tool-schemas.md` it should work much better.

```
I would like to create an application that allows me to query salesforce in both free text and as SOQL. When the query runs successfully I'd like to see the results in a grid. I would also like to be able to export this grid to CSV.

Use the following tools:

querySalesforce
querySalesforceWithFreeText
```

The above prompt generated the below UI, but when I run it, I get the below error.



For whatever reason the app that it built didn't use

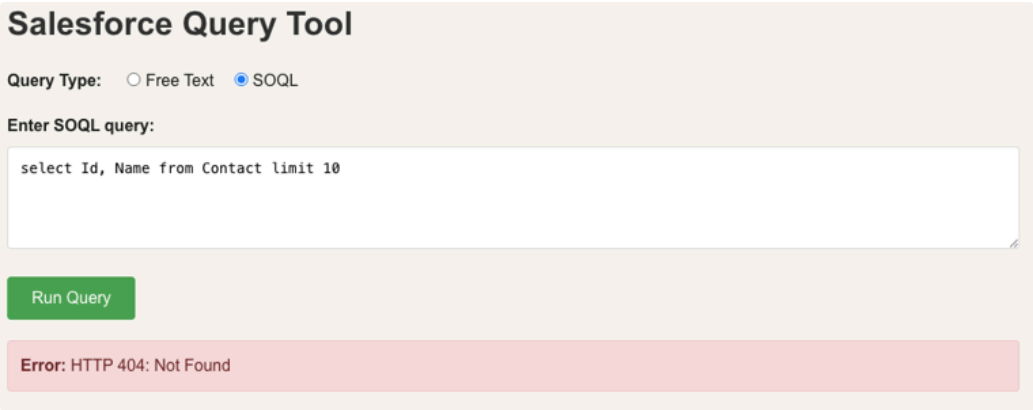
```
const { apiClient } = useMicroAppContext();
```

so I prompted it to do so (this occurred during an early beta of the MicroApp creation process, so it likely won't be a problem for you)

```
this app isn't working you should be using

const { apiClient } = useMicroAppContext();
```

the next time I ran it, I got



Looking into the error, I see that it was using the wrong tool name (`querySalesforce`) so I prompted again

```
the tool names are wrong and should be named

zoominfo-salesforce-service-querySalesforce
or
zoominfo-salesforce-service-querySalesforceWithFreeText
```

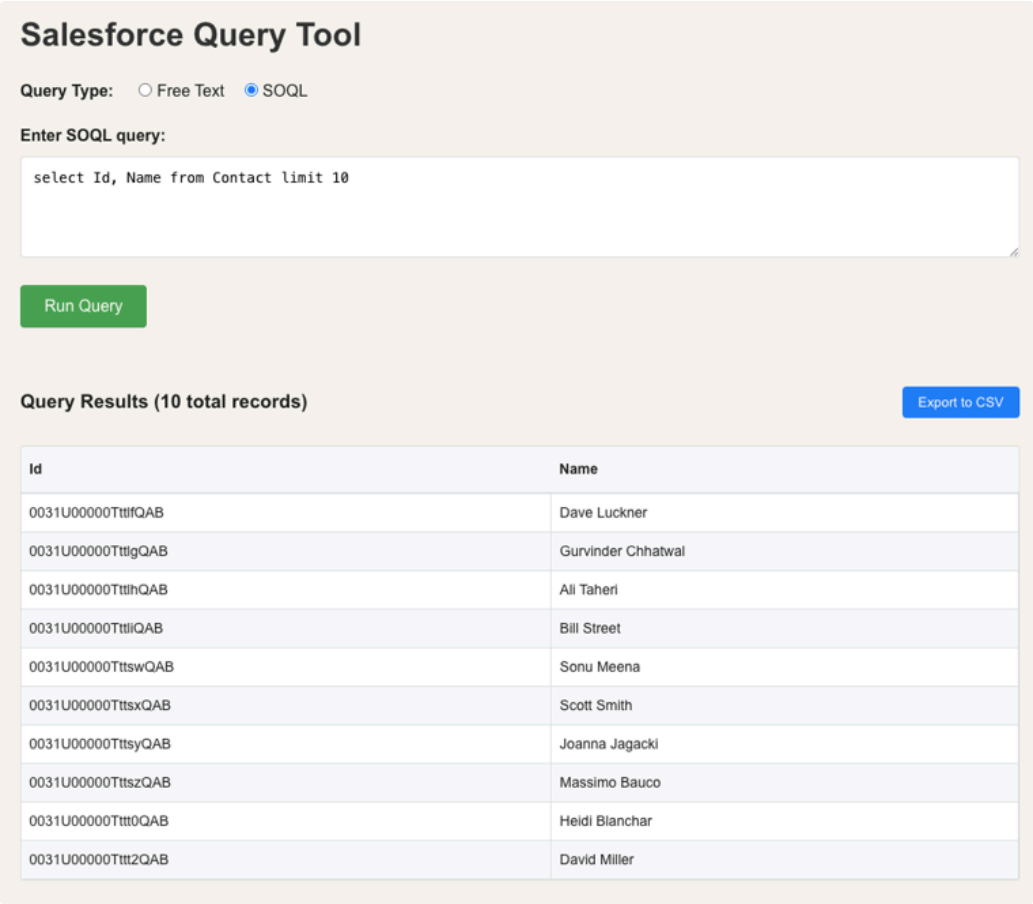
I ran it again and I see the following

When I look at the chrome debug tools, I see that results are coming back. Maybe it doesn't understand how to read the results.

```
it says No results returned from query, but I see results.

```
{
    "structuredContent": [
        {
            "query": "select Id, Name from Contact limit 10",
            "totalSize": 10,
            "records": [
                {
                    "Id": "0031U00000TttlfQAB",
                    "Name": "Dave Luckner"
                },
                {
                    "Id": "0031U00000TttlgQAB",
                    "Name": "Gurvinder Chhatwal"
                }
            ],
            "nextPageData": null
        }
    ]
}
```
```

Yay it worked!



Testing Your Code

The above actually only kind of worked. Export to CSV is still broken + Free Text is as well. The moral of this story is to be sure to test ALL the functionality you're providing including various scenarios/edge cases. If you need help brainstorming things that could go wrong, feel free to ask Cline (in plan mode) or ZI Chat for ideas. It's way better to catch problems early on than to get pinged by a ton of people!

Running your Code

When Cline is finished coding you can run the following command. You can always interrupt cline when it starts doing weird things and roll back to an earlier point in time.

```
npm run dev
```

Deploying your MicroApp

Congratulations, you've built something useful. You've tested it thoroughly (right?). You are ready to let others see your masterpiece. It's time to "ship it".

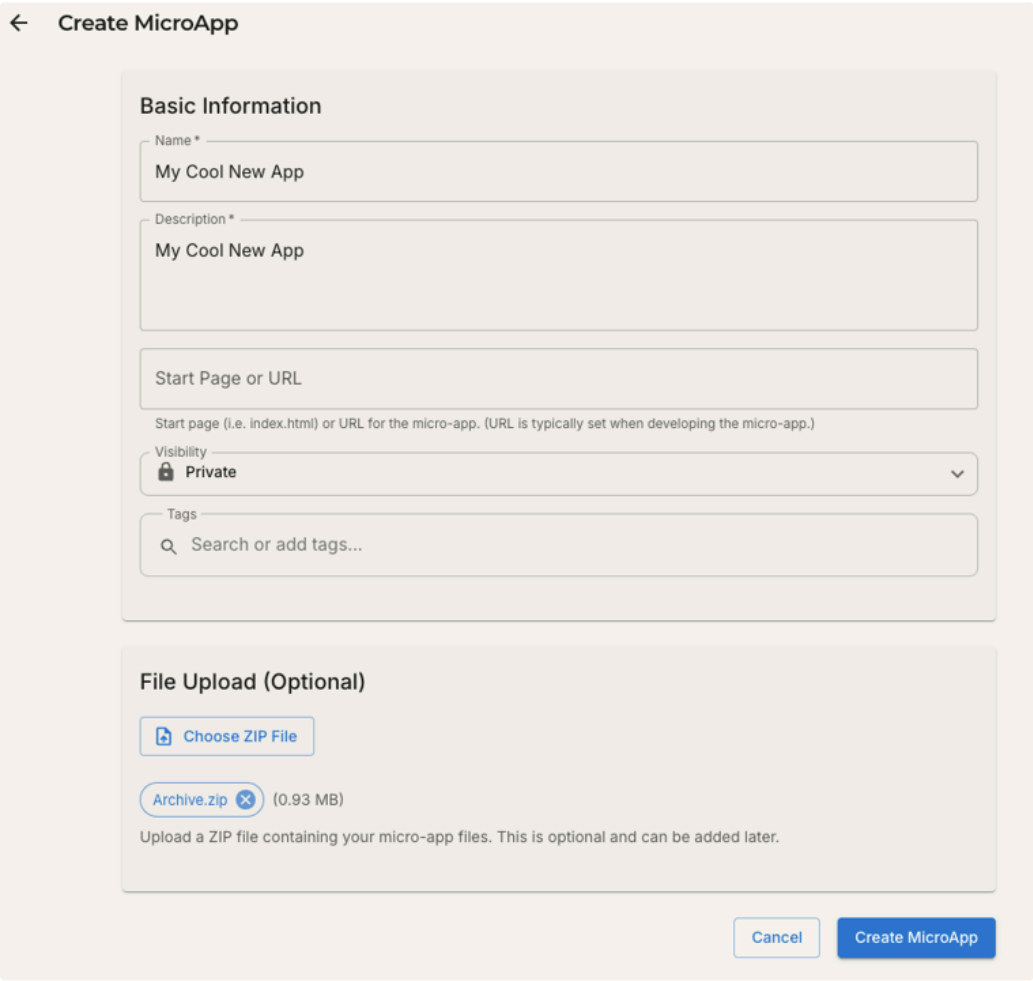
Build your application via this command in your IDE:

```
npm run build
```

You should see something like the below

```
📦 Building micro-app for production...
vite v5.4.20 building for production...
✓ 1161 modules transformed.
dist/index.html                                          0.44 kB
gzip:   0.31 kB
<---------------a bunch of files-------------------->
✓ built in 1.46s
✅ Build completed successfully!
Output directory: <some path>/dist
🗜️ Compressing build output...
✅ Created '<some path>/build_1.0.0.zip' (0.96 MB)
```

A zip file with your microapp is automatically created. Make note of the path since you will be uploading it on the Create MicroApp screen.



Additional Help

This video may help you if you get stuck. Please note, that this may have some legacy steps that are no longer needed.

Short guide on deploying your app and troubleshooting issues:

Seamless video recording & sharing

Passcode: ??xEn0jx

Supporting your MicroApp

Congratulations! You've built something amazing and people are using it. You may start to get a lot of questions about your app.

- How do I use it?
- Can you add feature XYZ?
- I found a bug when are you going to fix it?
- Can you make it run on my flip phone?

Don't worry, this is a normal part of app development. If people are asking for changes or new features, that means you've provided them with value! It's your app so you're in the driver's seat. If a request is too narrow for it to help others, maybe recommend someone makes their own MicroApp and show them how! Maybe push your source code to a git repo so you can collaborate and make an even better MicroApp. Happy developing!

Updating the MicroApp Framework

From time to time, new features will be added to the MicroApp framework. You will be notified via the `#zi-chat-microapps` channel that you may run this step to take advantage of new features. Depending on the scope of the framework change, there may be other steps you need to do, but if that happens it will be communicated via the slack channel.

> ⚠ It is strongly recommended you use git (doesn't have to be github) prior to making any major changes to your MicroApp.

To take advantage of new micro app features run the following commands:

```
npm update @zoominfo-chat/micro-app-core

# if you want to update your cline rules docs (it will ask you if you want to overwrite each one)
npm run update-docs
```

If `npm run update-docs` doesn't work, you will need to add it to your `package.json` file. It should look something like below:

```
  "scripts": {
    "dev": "micro-app dev",
    "build": "micro-app build",
    "preview": "vite preview",
    "update-docs": "micro-app update-docs"
  },
```

When you run this update you should see something similar to the below:

```
added 75 packages, changed 1 package, and audited 333 packages in 11s
<other stuff>
```

> It is not uncommon for new features to require updated .clinerules. For safety, we do not update your .clinerules directory (or any other non node_modules directory) since you may have made changes. If you would like the latest .clinerules, you can create a new micro app and manually copy over any files yourself. We may make this easier in a future build.
>
> npx @zoominfo-chat/create-micro-app some-dummy-micro-app-name

Rate Limits

The MicroApp framework gives you a lot of power, but we all have a responsibility to manage costs and maintain the health of the ZI-Chat application. As such, we have implemented rate limits to keep costs from getting out of control in the event of a bug or an overly aggressive MicroApp. Below are the current rate limits:

API Call	Limit (max requests per minute per user)
`callTool`	30
`getTools`	5
`callZoomInfoApi`	10
`executePromptStream` / `executePromptSync`	10
`getAgents`	5
`getAgent`	5
`executeAgentSync` / `executeAgentAsync`	10
`getAgentExecutionStatus`	100
`getUsers`	10
`getAvailableMicroApps`	10
`dataStoreExecuteQuery`	120

`snowflakeExecuteQuery`	30
`storeUserData` / `updateUserData` / `deleteUserData` / `listUserData` / `getUserData`	60

When you successfully call a rate-limited MicroApp endpoint, you will receive the following headers:

Header name	What it means
`x-ratelimit-limit`	The maximum number of requests you're allowed to make in the current window.
`x-ratelimit-remaining`	How many requests you have left before you hit the limit.
`x-ratelimit-reset`	Number of seconds before the request bucket is refilled

If your application is being rate limited, you will receive a 429 http status code and the below header:

Header name	Header value
`retry-after`	number of seconds until the request bucket is refilled

It is your app's responsibility to handle rate limits gracefully. If you find that these rate limits are holding your app back from greatness, please let us know in the `#zi-chat-microapps` channel,

Working with Snowflake Data

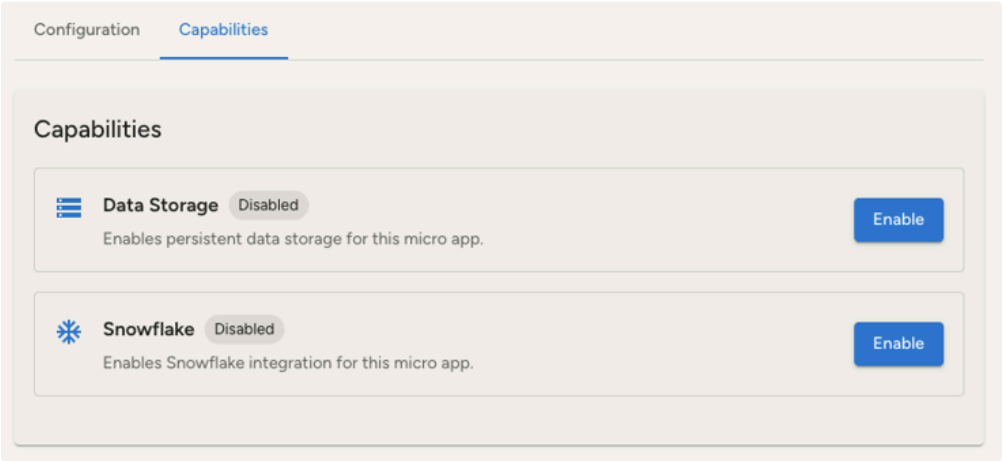
⚠ This feature is in preview be sure to update the framework so you have access to these features!

The MicroApp snowflake capability gives your MicroApp access to 2 isolated snowflake schemas (one for dev and one for production).

> Please see: ZI Chat - Microapp Snowflake Storage - Usage Guide & Demo for a full usage guide, troubleshooting, and E2E demo using Snowflake.

Enabling Snowflake

1. Navigate to your micro app.
2. Select “Edit”
3. Select the “Capabilities” tab and click “Enable” next to the “Snowflake” capability. This may take up to ~2 minutes to enable.



4. Once it is enabled, you may login to Snowflake via your Okta tile (put in a service now ticket if you don’t have access). You will now have access to `c.MICROAPP_###` and `PRD_ZI_CHAT.MICROAPP_###_DEV` schemas. To determine which schema belongs to your micro app, go to your micro app in the UI and look at the URL. The URL is in this format: `https://..../micro-apps/{workspaceId}/{microAppId}`. Now that you can access your micro app via an IDE, you can start working on it.

snowflake.yaml

Since MicroApps don’t have a back-end, you must store all your queries in the `snowflake.yaml` file. This file allows you to write parameterized sql statements (example here) that can be executed via the front end using the `snowflakeExecuteQuery` api call. Please look at the Snowflake.tsx example and/or cline rules to help you better understand how to work with this file.

Bulk Loading

For scenarios where you need to insert or update many records at once (hundreds or thousands), use a bulk operation with a JSON parameter instead of making individual API calls. This dramatically improves performance and avoids rate limiting.

Use Cases:

- Uploading a spreadsheet of contacts
- Batch updating multiple records
- Initial data imports

Important: Parameter Syntax Conflicts

- You must not use `PARSE_JSON` for registering queries. `TRY_PARSE_JSON` is required to avoid errors during registration.
- When using `TRY_PARSE_JSON` with the MicroApp parameter system, you must avoid syntax conflicts:

❌ **WRONG** - Colon operator (`:`) in JSON field access conflicts with parameter syntax:

```
-- This will fail validation!
SELECT value:test_col_1 FROM TABLE(FLATTEN(TRY_PARSE_JSON(:json_data)))
```

❌ **WRONG** - Double colon casting operator (`::`) also conflicts:

```
-- This will also fail validation!
SELECT value:test_col_1::VARCHAR FROM
TABLE(FLATTEN(TRY_PARSE_JSON(:json_data)))
```

✅ **CORRECT** - Use bracket notation for JSON field access and function-based casting:

```
SELECT
  TO_VARCHAR(value['test_col_1']) AS test_col_1,
  TO_NUMBER(value['test_col_2']) AS test_col_2
FROM TABLE(FLATTEN(TRY_PARSE_JSON(:json_data)))
```

Example: Bulk Insert/Update Query

```
queries:
  - name: bulk_merge_records
    sql: |
      MERGE INTO target_table t
      USING (
        SELECT
          TO_VARCHAR(value['id']) AS id,
          TO_VARCHAR(value['name']) AS name,
          TO_VARCHAR(value['email']) AS email,
          TO_NUMBER(value['age']) AS age,
          TO_TIMESTAMP(value['created_at']) AS created_at
        FROM TABLE(FLATTEN(TRY_PARSE_JSON(:json_data)))
      ) s
      ON t.id = s.id
      WHEN MATCHED THEN
        UPDATE SET
          t.name = s.name,
```

```
          t.email = s.email,
          t.age = s.age
      WHEN NOT MATCHED THEN
        INSERT (id, name, email, age, created_at)
        VALUES (s.id, s.name, s.email, s.age, s.created_at)
    parameters:
      - name: json_data
        type: VARCHAR
```

Example: TypeScript Implementation

```typescript
interface BulkRecord {
  id: string;
  name: string;
  email: string;
  age: number;
  created_at: string;
}
async function bulkUploadRecords(records: BulkRecord[]) {
  // Convert array to JSON string
  const jsonData = JSON.stringify(records);
  // Execute bulk operation
  const result = await apiClient.snowflakeExecuteQuery(
    "bulk_merge_records",
    { json_data: jsonData }
  );
  if (result.success) {
    console.log(`Successfully processed ${records.length} records`);
    console.log(`Rows affected: ${result.data.rowCount}`);
  } else {
    console.error("Bulk upload failed:", result.error);
  }
}

// Usage example with 500 records
const records: BulkRecord[] = [
  { id: "1", name: "John Doe", email: "john@example.com", age: 30, created_at: "2025-01-26T10:00:00Z" },
  { id: "2", name: "Jane Smith", email: "jane@example.com", age: 28, created_at: "2025-01-26T10:01:00Z" },
  // ... 498 more records
];

await bulkUploadRecords(records);
```

Incremental Fetching Data

You may want to paginate through results instead of fetching all results in a single call. Right now, there are hard limits on number of records returned from Snowflake (10,000). If you need more (or want to incrementally display data), you will want to follow this structure:

```
queries:
  - name: incremental_fetch
    sql: |
        SELECT
          * FROM TABLE ORDER BY COLUMN_NAME LIMIT :limit OFFSET :offset
    parameters:
      - name: limit
        type: INTEGER
      - name: offset
        type: INTEGER
```

⚠ You must use an `ORDER BY` clause or the results will be non-deterministic: LIMIT / FETCH | Snowflake Documentation

Example usage:

```
async function fetchAllData() {
  const limit = 1000;
  let offset = 0;
  let allResults = [];

  while (true) {
    const result = await apiClient.snowflakeExecuteQuery(
      "incremental_fetch",
      { limit, offset }
    );

    if (result.data.rows.length === 0) {
      break;
    }

    allResults = allResults.concat(result.data.rows);
    offset += limit;
  }

  return allResults;
}

// Usage
const data = await fetchAllData();
console.log(`Fetched ${data.length} total rows`);
```

Schema Management

- For now, you are responsible for managing schema between your dev and prod schemas.

Backup/Restore

- Snowflake data is automatically backed up. You may use snowflake's time travel functionality to restore data.

Monitoring

- For now, we do not have a public way for you to monitor your query performance. This is something we may add in the future if the need arises.

Data Security

- **You are not allowed to store any sensitive data in micro-apps** (i.e. PII, HIPAA, SSN, Credit Card Numbers, etc).

Working with Postgres Data Store

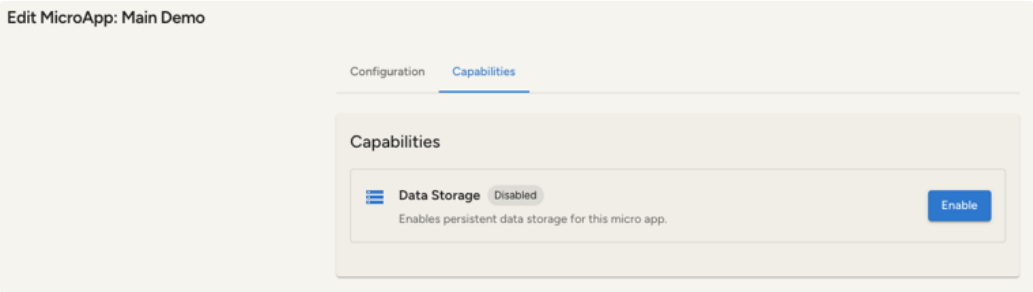
⚠

> This feature is in preview be sure to update the framework so you have access to these features!

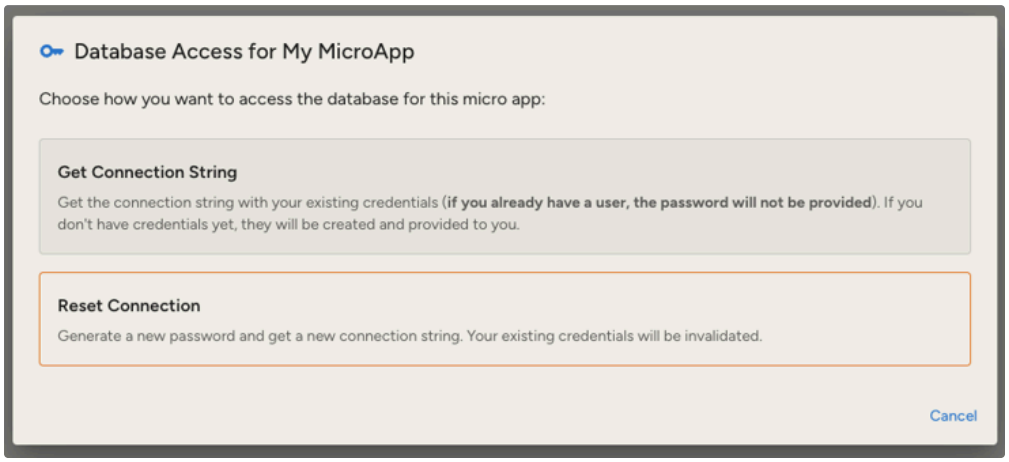
The MicroApp Data Storage capability gives your MicroApp access to 2 isolated postgres schemas (one for dev and one for production).

Enabling Data Store

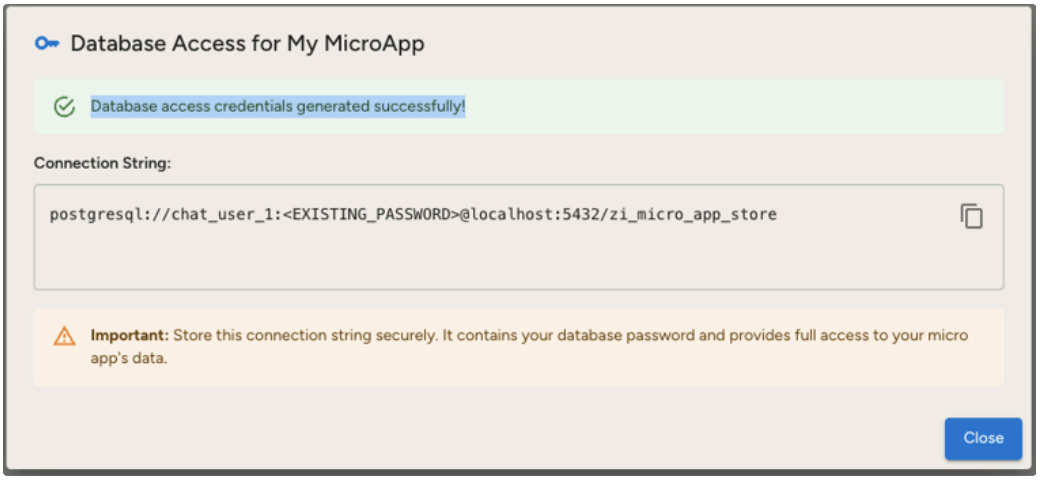
1. Navigate to your micro app.
2. Select “Edit”
3. Select the “Capabilities” tab and click “Enable” next to “Data Storage”



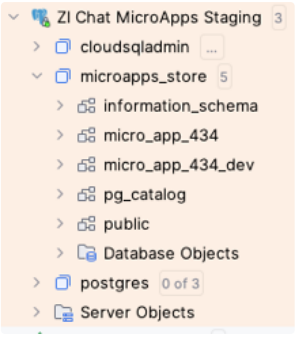
4. While you’re here, you may want to click “Get Database Access” and capture the connection string so you can create schema, write queries, and migrate data.



5. Click “Get Connection String” you should see something like below (it will have an IP address like 10.x.x.x instead of localhost). You will need to use ZScaler to access this database. (If you forget your password, you can click “Reset Connection” to get a new one.)



6. Save this connection string in your favorite Database management IDE.
7. Once you've set up your IDE, you will see a lot of different schemas (postgres has some limitations around what we can hide without making it hard for database migration tools to function). You should see something like below.



In the above example, you will see 2 schemas: `micro_app_434` and `micro_app_434_dev`. These are the schemas for `microAppId` 434. To determine which schema belongs to your micro app, go to your micro app in the UI and look at the URL. The URL is in this format: `https://..../micro-apps/{workspaceId}/{microAppId}`. Now that you can access your micro app via an IDE, you can start working with it.

Working with Data (Simple Mode)

When a new schema is created, by default it automatically creates 2 tables. `global_data` and `user_data`. These tables are containers for storing arbitrary JSON payloads (separated by a field called `type`). This isn't a particularly efficient way of storing data, but it will work for many simple micro app use cases.

We have APIs for CRUD on `user_data` (you will have to use data-store.yaml for `global_data`). All calls to this table are automatically scoped to the signed in user (using

postgres row level security). For information on how to interact with these tables, please look at the DataStorage.tsx example and/or cline rules.

Working with Data (Advanced Mode)

If you are comfortable with SQL or have a more complex than a few thousand row use case, you should create tables on your own. Doing this is outside the scope of this document, but essentially you would just run CREATE TABLE statements.

> If you are creating tables, be sure to set the role:
> Dev:
> `set role micro_app_<YOUR_MICRO_APP_ID_GOES_HERE>_dev_admin`
> Prd:
> `set role micro_app_<YOUR_MICRO_APP_ID_GOES_HERE>_admin`

data-store.yaml

Since MicroApps don't have a back-end, you must store all your queries in the `data-store.yaml` file. This file allows you to write parameterized sql statements/queries (example here) that can be executed via the front end using `dataStoreExecuteQuery` api call. Please look at the DataStorage.tsx example and/or cline rules to help you better understand how to work with this file.

Schema Management

- For now, you are responsible for managing schema between your dev and prod schemas. We recommend taking a look at flyway.
- Be sure to take backups of your data before migrations.

Backup/Restore

- The micro apps database is automatically backed up, but those backups are for emergency restoration of all micro apps only.
- Unfortunately, we do not have a good way for you to perform backup/restore while we're in preview mode.

Performance

- If you think you will end up with more than a few thousand rows, we recommend creating your own tables and adding appropriate indexes.
- If we start running into performance issues in the future we may impose query timeouts.

Monitoring

- For now, we do not have a public way for you to monitor your query performance. This is something we may add in the future if the need arises.

Data Security

- **You are not allowed to store any sensitive data in micro-apps** (i.e. PII, HIPAA, SSN, Credit Card Numbers, etc).
- When writing queries that don't use the `user_data` table, be careful not to "leak" a users's data to other users. You can use the `:chatUserId` (no need to pass it in your `dataStoreExecuteQuery` payload).

Getting Help

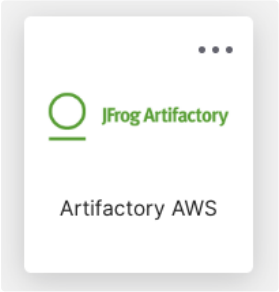
Are the AI vibes not working for you? Please reach out to `#zi-chat-microapps` channel for support. The ZI Chat developers are generally happy to help, but you probably want us adding more and more features to the main app instead of helping with MicroApps all day. The chat channel should be your first and only destination for these types of questions.

Troubleshooting

There are a lot of things that can go wrong when you're developing. Using the Chatbot to help you figure out the issue is a good first step. However there are some issues that we've seen that are tough to figure out, so they are documented below.

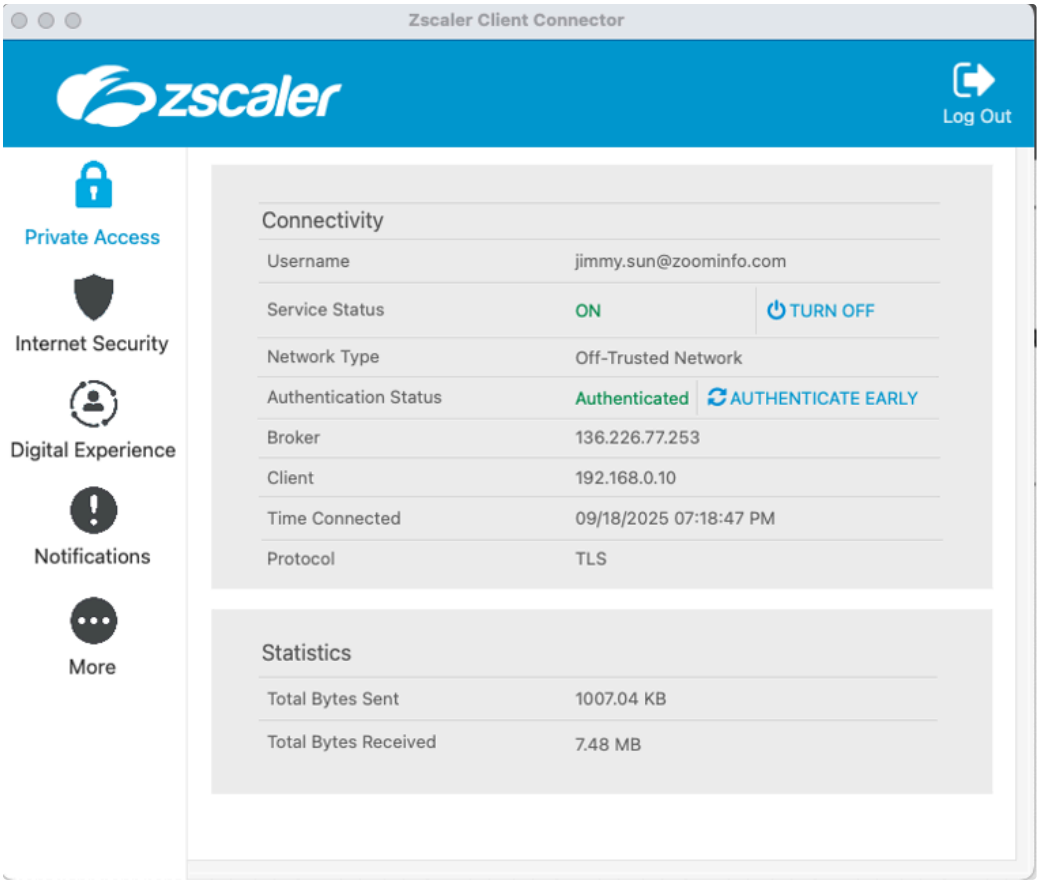
NPM

Once your service now ticket has been approved you should see this Okta Tile



WITH Zscaler Private Access

If you have Zscaler specifically the `Private Access` then make sure you are signed in to the network.



- No need to login as authentication happens through zscaler

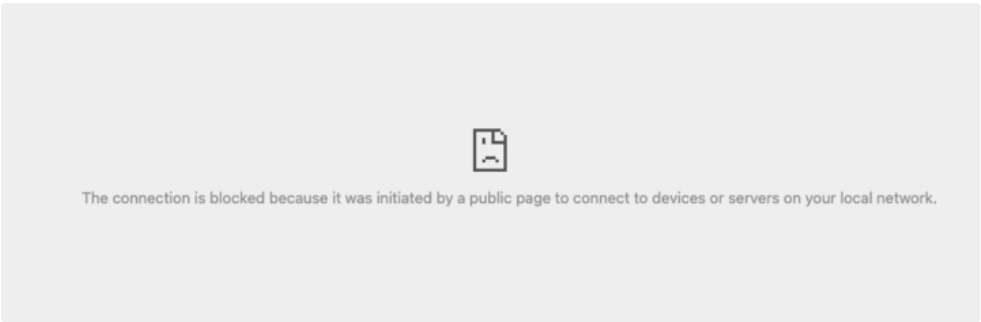
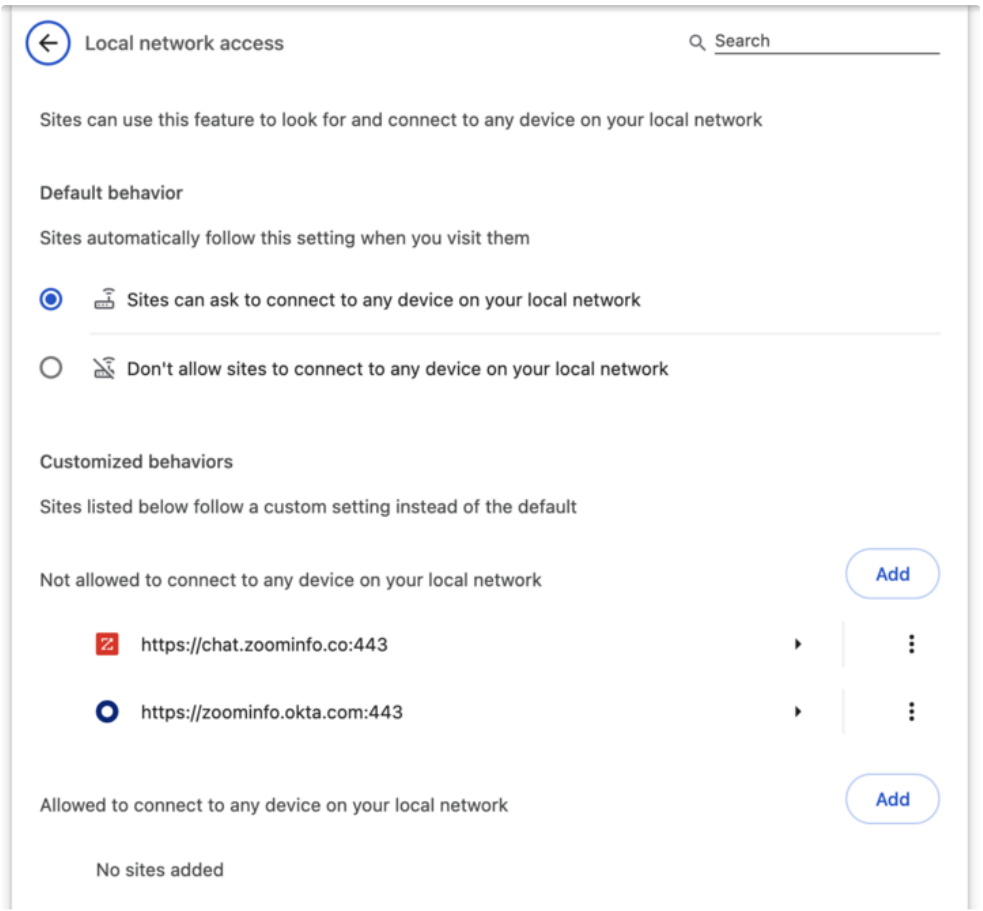
WITHOUT Zscaler Private Access

- run `npm login --registry=https://zoominfo.jfrog.io/artifactory/api/npm/virtual-npm/ --auth-type=web`
- Update your `.npmrc` file with these contents

```
registry=https://zoominfo.jfrog.io/artifactory/api/npm/virtual-npm/
```

Developer Access Not Working

If you see a message like below, ZIChat may be blocked from accessing your local machine via iframe.



To fix this, put `chrome://settings/content/localNetworkAccess` into your chrome address bar. If you see ZI Chat's URL in the "Not allowed to connect to any device on your local network" (like it is in the below screenshot). Remove it from that list.